

Based on the panel's feedback during your capstone defense, the suggestions can be categorized into technical revisions, functional focus shifts, and documentation standards.

The panel emphasized that while you have a working management tool, it currently lacks the specialized features and is required for a true **Computer System Servicing (CSS)** Learning Management System (LMS).

1. Administrative & Workflow Revisions

These are changes to how the system handles users and data to move away from manual bottlenecks.

- **Revisions to Change:**
 - **Remove Manual Registration Approval:** Stop requiring the admin to manually verify every student account; it is impractical for large populations.
 - **Eliminate "Unsubmit" Button:** Remove the option for students to unsubmit assignments to ensure finality, or replace it with a "Are you sure?" confirmation.
 - **Modify Invitation Process:** Instead of inviting students one by one, allow for "mass selection" or group-based enrollment.
- **New Focus for Development:**
 - **CSV/Bulk Data Upload:** Focus on allowing teachers to upload a **CSV file** (class list) to enroll entire sections at once.
 - **LRN-Based Automation:** Use the **Learner Reference Number (LRN)** as a unique ID for automated verification. If a student's LRN matches a pre-uploaded master list, they should be approved automatically.
 - **Removal of Email Dependency:** If LRN and CSV automation are implemented, the panel suggested you could potentially remove the Gmail notification step entirely to streamline the process.

2. Core System Functionality (LMS vs. Management)

The panel felt the system currently behaves more like a "student management system" and needs to become a dynamic LMS.

- **Revisions to Change:**
 - **Static to Dynamic Modules:** Move away from "pre-built" or hardcoded modules that only the Super Admin can manage.
- **New Focus for Development:**
 - **Teacher-Led Content Creation:** The primary focus must be on **enabling teachers to create, add, and manage their own modules** and subjects from scratch so the system can evolve with curriculum changes.

3. Curriculum Specificity (The "CSS" Focus)

This was a major point—differentiating your system from common tools like Google Classroom.

- **Revisions to Change:**
 - **Differentiate from Google Classroom:** Avoid just being a file-sharing and quiz platform; the panel asked what makes this unique to CSS.
- **New Focus for Development:**

- **Interactive Simulations:** Implement **CSS-specific interactive activities**, such as drag-and-drop simulations for **assembling computer parts**, identifying motherboard components, or network configurations.

4. Technical Diagrams & Architecture

These are critical fixes for your technical documentation to ensure it matches standard logic and your actual implementation.

- **Revisions to Change:**
 - **Flowchart Symbols:** Every process box must have only **one exit arrow**. Use **diamond-shaped decision boxes** for any step with multiple outcomes (like user level authentication).
 - **Single Ending:** Ensure the flowchart has only **one "End" point**.
 - **DFD Numbering:** Keep numbering consistent across all users (e.g., if 1.0 is Login for the admin, 1.0 should also be Login for the teacher).
 - **Declutter DFDs:** Reduce the number of inputs and outputs entering a single process circle to make them readable.
- **New Focus for Analysis:**
 - **Clarify Network Model:** You must decide and clearly state if the system is **web-based (online)** or **LAN-based**. Since it uses email and browser access, the panel noted it should be treated as a web-based system deployed online.
 - **Existing Process Analysis:** Before showing your proposed system's Use Case, you must first document and analyze the **existing manual process**.

5. Documentation & UI/UX Standards

General improvements to the professional quality of your research paper and interface.

- **Revisions to Change:**
 - **Update Literature:** Remove all references from 2021; your sources must be from **2022 onwards**.
 - **Redesign UI Colors:** The panel found the green color too bright and the layout too "loose".
 - **Restructure Introduction:** Use a **deductive approach**—start with the general state of the digital age and LMS technology before focusing on your specific school's problems.
- **New Focus for Content:**
 - **Demographic Profile:** Include the **school's population data** (number of students and teachers) to justify why the system needs this specific scale.
 - **Justify Methodology:** Provide a stronger rationale for using **Agile**, citing studies where it was successful for similar projects.

6. From Management to "Learning"

The panel noted that your system is currently more of a **student management tool** than a true LMS.

- **What this means:** Your study is no longer just about approving accounts and viewing files. It must focus on **"Learning"**, which requires giving teachers the power to **create and customize dynamic modules** from scratch rather than just using pre-built ones.

Beyond the major shifts in automation and CSS-specific features already discussed, the panelists made several other specific points regarding your documentation, technical infrastructure, and professional conduct that you should address.

1. Technical Infrastructure & Deployment

The panel was highly confused by your network model and how the system would actually be used in the school.

- **Infrastructure Reality Check:** You must consider the school's actual logistics. Do they have a local area network (LAN) or reliable Wi-Fi? You need to provide a clear **implementation plan** that explains how students will access the server.
- **Conflicting Models:** Your documentation mentions a "Star Topology" and "LAN," but your demo used email notifications which require internet access. One panelist noted that your "Deployment Diagram" did not match the rest of your paper, suggesting the different sections of your documentation were not properly coordinated.
- **Database Specification:** You need to explicitly define your database in the network model, as the current version stops at the server level and doesn't show where data is stored.

2. Research & Documentation Standards

There were several "non-negotiable" academic requirements mentioned:

- **Statement of the Problem:** One panelist suggested you might need to remove or significantly reformat this section if it doesn't follow their required template.
- **Literature Grouping (RRL):** Instead of just listing "Foreign" and "Local" literature, you were instructed to group your sources into **sub-themes** based on common topics (e.g., "User Experience in LMS" or "Automated Grading").
- **ISO Standards:** Your objective mentions evaluating the system based on **ISO 25010** software quality standards (functionality, reliability, etc.). The panel will expect you to show how you actually measured these.
- **Demographic Profile:** You specifically missed providing the population data for Paliparan 3 SHS (number of students and teachers), which is necessary to justify the scale of your system.

3. Specific UI and Content Critiques

- **Non-CSS Content:** The panel criticized your use of an "Introduction to Philosophy" module during the demo. They argued that for a CSS-specific system, your demo content should strictly be computer-related (e.g., Java, hardware assembly) to prove the system works for its intended purpose.
- **"Hotdog" Filenames:** During the demo, a file named "hotdog" was used as an example. The panel noted that the UI should show formal file names and perhaps allow for file renaming or clear identification of what was submitted.
- **UI Layout:** Aside from the color, the layout was described as **"too loose,"** suggesting there is too much empty space that could be better organized.

4. Teacher Management Workflow

The panel felt the teacher's role was currently too passive.

- **Load Management:** A teacher doesn't just "view" a class; they receive a "load" from the principal. Your system needs to reflect this scenario: a teacher gets a class list, sets up their subjects, and uploads modules specifically for those sections.
- **Standardization vs. Customization:** There was a debate about whether modules should be "standard" (set by the school) or if every teacher can create their own unique sets. You need to decide if your LMS supports a **standardized curriculum** or allows for individual teacher variations.

The implementation of **LRN-based automated verification** was suggested by the defense panel as a way to replace the current manual approval process, which they deemed impractical for large student populations.

Based on the sources, the workflow for this automated verification would function as follows:

- **Data Import as the Basis:** The administrator (admin) or teacher would first **import a master list** of student data into the system, likely through a **CSV file**. This list would serve as the "dummy data" or source of truth, containing every student's name and their unique **Learner Reference Number (LRN)**.
- **Automatic Matching:** When a student attempts to register or log in, the system would require them to provide their name and LRN. The system would then **automatically compare** these inputs against the pre-uploaded records.
- **Instant Approval:** If the student's name and LRN match the records provided by the admin, the account is **approved automatically** by the system. This removes the "bottleneck" of requiring an administrator to manually counter-check every single registration request against a separate class list.
- **Simplified Login and Credentialing:** One panelist suggested that students could potentially log in using their LRN and an initial password provided by the school. This shift would effectively **eliminate the need for email-based registration** and manual confirmation, as the LRN itself acts as the primary verification factor.

This change is intended to **standardize the enrollment process** and significantly reduce the administrative workload at the start of the school year.

Based on the defense transcript and our conversation history, your project has transitioned from a manual "student management" tool to a dynamic, automated, and trade-specific Learning Management System (LMS).

Before the Defense: The Initial Focus

Before receiving the panel's feedback, your system was at approximately **50% completion** and functioned primarily as a centralized digital repository for existing materials.

- **Manual Workflows:** Students registered using personal email accounts, and administrators had to manually review and approve each account one by one.
- **Static Learning Content:** The system relied heavily on **pre-built (static) modules** managed by the Super Admin, which teachers and students could view but not significantly customize.
- **Communication & Access:** The primary goal was to provide a dedicated platform to replace scattered materials and informal communication via Messenger.
- **Infrastructure Confusion:** The documentation described a local **Star Topology (LAN)**, yet the system required internet access for its email notification and browser-based features.
- **General Content:** The system included general academic subjects, such as "Introduction to Philosophy," which was not specific to the Computer System Servicing (CSS) trade.

After the Defense: The New Focus (Panel Suggestions)

The panel redirected your study to focus on **scalability, automation, and specialization** to ensure it provides unique value compared to existing platforms like Google Classroom.

- **Administrative Automation:**
 - **CSV Bulk Uploads:** Instead of manual invitations, the focus is now on allowing teachers to upload **CSV files** to enroll entire sections simultaneously.
 - **LRN-Based Verification:** You are now tasked with implementing automated approval by matching a student's name and **Learner Reference Number (LRN)** against a pre-loaded master list.
 - **Removing Email Dependency:** By using LRNs for automated access, the panel suggested you could potentially remove the "extra step" of email registration and confirmation.
- **Dynamic Learning Experience:**
 - **Teacher-Led Creation:** The focus has shifted to ensuring **teachers can create and manage their own modules** and subjects from scratch to keep the curriculum current.
- **CSS Trade Specialization:**
 - **Interactive Simulations:** To differentiate your system, you must now include **CSS-specific interactivity**, such as drag-and-drop simulations for assembling computer hardware or identifying motherboard components.
- **Technical & Academic Rigor:**

- **Correcting Diagram Logic:** You must revise your flowcharts to use correct symbols (one exit arrow for processes, diamond boxes for decisions) and ensure consistent numbering across all Data Flow Diagrams (DFDs).
- **Updating Research:** All literature must be updated to **2022 or later**, and you must include a school **demographic profile** to justify the system's scale.
- **Deployment Clarification:** You must formally document the system as an **online web-based application** rather than a local LAN setup.

Summarized Path Forward

In short, your focus has moved from **manually managing users and viewing static files** to creating a **fully automated, teacher-driven platform** that uses **trade-specific simulations** to revolutionize how CSS is taught at your school.

Import Master Lists via CSV (*Admin*)

Think of this as the first-time setup of all students in the school.

How it works:

1. The Admin downloads or prepares a CSV file (basically a spreadsheet saved as .csv) containing all student information — like name, student ID, grade level, section, etc.
2. The Admin goes to the Admin Dashboard and clicks an "Import" button
3. They upload the CSV file into the system
4. The system reads each row of the file and automatically creates a student account for each one
5. All students are now registered in the system at once — no need to add them one by one

Think of it like: Submitting the school's official enrollment list to the system in bulk.

Bulk Enroll Students via CSV (*Teacher*)

This is for enrolling already-registered students into a specific class.

How it works:

1. The Teacher prepares a CSV file containing the list of students they want to enroll in their class (usually just student IDs or names)
2. The Teacher uploads this file inside their class/module
3. The system matches each student ID from the file to existing student accounts
4. Those students are then automatically enrolled into the teacher's class

Think of it like: A teacher submitting a class roster so the system knows which students belong to their subject.

While the primary focus of your system is on **Computer System Servicing (CSS)** students, the question of which "subjects" are included was a major point of discussion during the defense.

According to the sources, here is the breakdown of how subjects are handled in your current revision:

1. Primary Focus: CSS-Specific Subjects

The panel emphasized that because your title specifically mentions **CSS students**, the system should prioritize **computing subjects** over general academic ones. They pointed out that if the system only hosts general subjects like "Introduction to Philosophy" (which was seen in your demo), it would be too similar to Google Classroom. To make the system unique, they recommended focusing on subjects that require **trade-specific interactivity**, such as:

- **Computer Hardware Assembly:** Simulations for "assembling and disassembling" computer parts.
- **Network Configuration:** Practice activities for networking tasks.
- **Hardware Identification:** Interactive "drag-and-drop" modules where students place parts on a virtual motherboard.

2. Current inclusion of General Subjects

During the defense, it was noted that the system currently includes subjects like **Introduction to Philosophy**. Your group's initial justification was that CSS students also have these general subjects in their curriculum. However, the panel argued that for the project to be considered "Specialized," it must focus on the technical competencies of the CSS strand.

3. Adaptability for Other Strands

While the project is currently limited to the CSS strand at Paliparan 3 Senior High School, you designed it with **adaptability** in mind.

- **Super Admin Capabilities:** The Super Admin has the authority to "create modules" and "manage subjects".
- **Future Scope:** This feature allows the school to eventually add subjects from other strands, such as **STEM** or **ABM**, if the curriculum changes or if the school wants to expand the use of the LMS.

Summary of the Revision

In your current revision, while you are gathering information on the general CSS curriculum, your development focus should be on **technical computing subjects**. The panel's goal is for you to move away from a generic "one-size-fits-all" LMS and instead create a tool that is **specifically designed for computing subjects** with specialized interactive features.